Reinforcement Learning

Clase 10

Ignacio Brevis

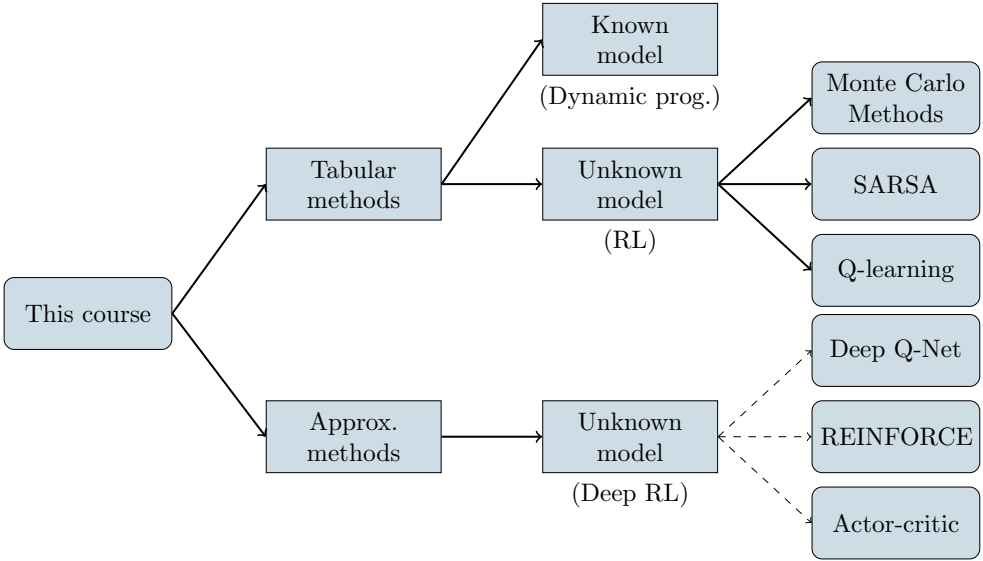
ignaciobrve@unisabana.edu.co

Universidad de La Sabana

21 octubre 2026



Repaso: Tabular vs approx. methods



Repaso: Tabular vs approx. methods

Métodos tabulares funcionan bien cuando

- Problemas de Gridworld (académicos)
- Juegos de mesa con "razonable" número de estados
- MDP con cientos o (unos pocos) miles de estados
- Problemas donde los estados se pueden enumerar explícitamente

Métodos aproximados entran en juego cuando:

- Espacios de estados continuos (articulaciones de robots, velocidades, lectura de sensores)
- Inputs de alta dimensión (juegos de Atari con 84×84 píxeles)
- Simulaciones físicas complejas (conducción autónoma)
- Espacios discretos muy grandes (Go, con $\sim 10^{170}$ posibles tableros)

Algunos tipos de funciones...

función lineal

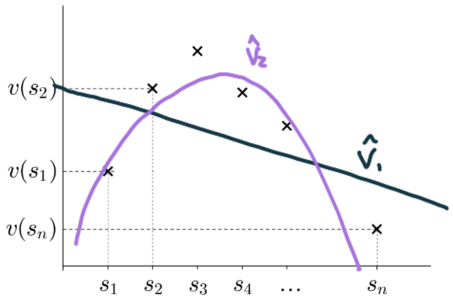
$\hat{v}_1(s;w) = w_1 + w_2 s$

$|w| = 2$

función cuadrática

$\hat{v}_2(s;w) = w_1 + w_2 s + w_3 s^2$

$|w| = 3$



Repaso: Function approximation

Diremos que la **aproximación es lineal** si se puede escribir de la siguiente forma

$$\hat{v}(s;w) = \sum_{i=1}^{d} w_i \phi_i(s) \qquad \left[= w^\top \phi(s)\right]$$

donde $w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}$ y $\phi(s) = \begin{bmatrix} \phi_1(s) \\ \phi_2(s) \\ \vdots \\ \phi_d(s) \end{bmatrix}$

Ejemplo: $\hat{v}(s;w) = w_1 + w_2 s + w_3 s^2$ es lineal en w pero no en s.

$$\hat{v}(s;w) = w = \begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} 1 \\ s \\ s^2 \end{bmatrix}$$

$\phi_1(s) = 1$
$\phi_2(s) = s$
$\phi_3(s) = s^2$

Repaso: Function approximation

Ejemplos no lineales: (c/r a w)

- $\hat{v}(s; w) = w_1 \exp(s) + w_2 \exp(-s)$ ✗
- $\hat{v}(s; w) = w_1 + w_2 s + w_1 w_2$ ✓
- $\hat{v}(s; w) = \exp(w_1) s + w_2$ ✓
- $\hat{v}(s; w) = \exp(-(s - w_1)^2) + \exp(-(s - w_2)^2)$ ✓
- redes neuronales! ✓

Más ejemplos:

función	lineal en s	lineal en w
$\hat{v}(s;w) = ws + 3$	✓	✓
$\hat{v}(s;w) = \frac{w_1}{1+e^{-s}} + \frac{w_2}{1+e^{-2s}} + \frac{w_3}{1+e^{-3s}}$	✗	✓
$\hat{v}(s;w) = \cos(w_1)s + \sin(w_2)s^2$	✗	✗
$\hat{v}(s;w) = \frac{s}{1+w_1} + \frac{2s}{1+w_2}$	✓	✗
$\hat{v}(s;w) = w_5 \tanh(w_1 s + w_2) + w_6 \tanh(w_3 s + w_4) + w_7$	✗	✗

Repaso: Neural Networks and expressivity

Neural network

A **fully-connected feedforward neural network** is a function $nn_\theta : \mathbb{R}^{N_1} \to \mathbb{R}^{N_{M+1}}$

$$nn_\theta(x) := L_M \circ L_{M-1} \circ \cdots \circ L_1(x),$$

where $L_i(x) = \sigma_i(W^{[i]}x + b^{[i]})$ is the i-th layer ($L_i : \mathbb{R}^{N_i} \to \mathbb{R}^{N_{i+1}}$).

- σ_i is the i-th **activation function**
- The collection of $W^{[i]}$ and $b^{[i]}$, $i = 1, \ldots, M$ are called **weights** and denoted by θ

Theorem 2. *Let σ be any continuous sigmoidal function. Then finite sums of the form*

$$G(x) = \sum_{j=1}^{N} \alpha_j \sigma(y_j^{\mathrm{T}} x + \theta_j)$$

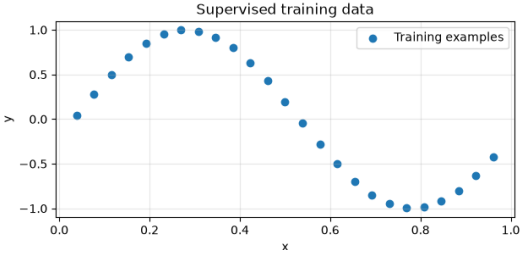
are dense in $C(I_n)$. In other words, given any $f \in C(I_n)$ and $\varepsilon > 0$, there is a sum, $G(x)$, of the above form, for which

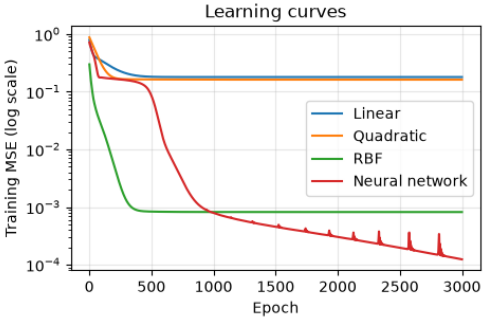
$$|G(x) - f(x)| < \varepsilon \quad \text{for all} \quad x \in I_n.$$

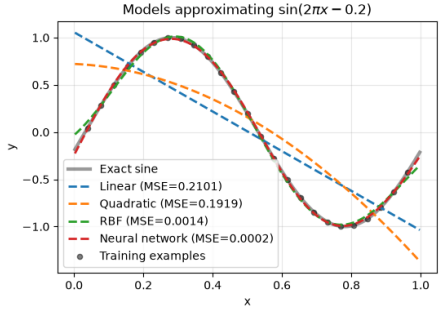
G. Cybenko, Approximation by superposition of a sigmoidal function, *Math. Control Signal Sys.* 1989.

Repaso: Function approximation

Ver código `Example_01-Function-approximation` y comparar la aproximación obtenida con los siguientes modelos:







Repaso: Value representation

Objetivo: Usar una función parametrizada (por w) para aproximar v y/o q:

estado	valor
s_1	$v(s_1)$
s_2	$v(s_2)$
$\vdots$	$\vdots$
s_n	$v(s_n)$

$\hat{v}(s; w)$ ← $|w| < u$

parámetros

	a_1	a_2		a_m
s_1	$q(s_1, a_1)$	$q(s_1, a_2)$	$\dots$	$q(s_1, a_m)$
s_2	$q(s_2, a_1)$	$q(s_2, a_2)$	$\dots$	$q(s_2, a_m)$
$\vdots$	$\vdots$		$\ddots$	$\vdots$
s_n	$q(s_n, a_1)$	$q(s_n, a_2)$	$\dots$	$q(s_n, a_m)$

$\hat{q}(s, a; w)$ ← $|w| < u \times m$

Obs: Este enfoque nos da un valor aproximado de la función de valor v_π y de la función de valor de acción q_π.

Repaso: Objective function

Dada una **política fija** π, definimos $v_\pi(s)$ como la función de valor (exacta) para cada $s \in \mathcal{S}$. Entonces, **queremos encontrar los valores de** w para los cuales $\hat{v}$ es la mejor aproximación de v_π. Para esto, utilizaremos el siguiente funcional:

$$J(w) = \frac{1}{2}\mathbb{E}\left[\left(v_\pi(S) - \hat{v}(S, w)\right)^2\right].$$

- El objetivo es encontrar el valor de w que minimiza $J(w)$.
- El valor esperado se calcula con respecto a la variable aleatoria S.
- Necesitamos calcular la distribución de probabilidad de S.

Repaso: States distribution

- **Distribución uniforme**: asume que cada estado tiene la misma probabilidad $1/n$ de ser elegido (dados n estados).

$$J(w) = \frac{1}{2} \sum_{s \in \mathcal{S}} \frac{1}{n} \left(v_\pi(s) - \hat{v}(s, w) \right)^2$$

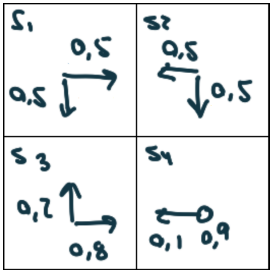
(no considera la dinámica del proceso de Markov)

- **Distribución estacionaria**: describe el comportamiento a "largo plazo" del proceso de decisión de Markov bajo una política π. En otras palabras, esta distribución, denotada por $d_\pi(s)$, describe la probabilidad de que el agente se encuentre en el estado s después de seguir una política durante un largo periodo.

$$J(w) = \frac{1}{2} \sum_{s \in \mathcal{S}} d_\pi(s) \left(v_\pi(s) - \hat{v}(s, w) \right)^2$$

Repaso: States distribution

La distribución estacionaria nos da la probabilidad de encontrarnos en un estado después de “infinitos” pasos



$$
\begin{aligned}
\begin{bmatrix}1 & 0 & 0 & 0\end{bmatrix} P_\pi &= \begin{bmatrix}0 & 0.5 & 0.5 & 0\end{bmatrix} \\
\begin{bmatrix}1 & 0 & 0 & 0\end{bmatrix} P_\pi^2 &= \begin{bmatrix}0.35 & 0 & 0 & 0.65\end{bmatrix} \\
\begin{bmatrix}1 & 0 & 0 & 0\end{bmatrix} P_\pi^3 &= \begin{bmatrix}0 & 0.175 & 0.24 & 0.595\end{bmatrix} \\
&\vdots \\
&= \begin{bmatrix}0.03 & 0.01 & 0.24 & 0.86\end{bmatrix}
\end{aligned}
$$

Repaso: States distribution

En el fondo, queremos estudiar qué sucede con

$$P_\pi^k$$

cuando $k \to \infty$ (esta rama de la matemática se conoce como sistemas dinámicos).

La distribución estacionaria se puede calcular resolviendo la ecuación:

$$d_\pi^\top P_\pi = d_\pi^\top$$

(Necesitamos conocer P_π para eso)

---

d_π se puede aproximar de la siguiente forma:

$$d_\pi(s) \approx \frac{n_\pi(s)}{\sum_{s' \in \mathcal{S}} n_\pi(s')}$$

Función objetivo:

$w_{k+1} = w_k - \alpha_k \nabla J(w_k)$

D. G.

$$J(w) = \frac{1}{2}\mathbb{E}\left[(v_\pi(s) - \hat{v}(s;w))^2\right]$$

$\min_w J(w)$

Función objetivo:

$$J(w) = \frac{1}{2}\mathbb{E}\left[(v_\pi(S) - \hat{v}(S; w))^2\right]$$

para minimizar J utilizaremos gradient descent:

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

$$\nabla_w J(w) = \nabla \frac{1}{2}\mathbb{E}\left((v_\pi(S) - \hat{v}(S;w))^2\right)$$

$$= \frac{1}{2}\mathbb{E}\left(\nabla (v_\pi(S) - \hat{v}(S;w))^2\right)$$

$$= \mathbb{E}\left((v_\pi(S) - \hat{v}(S;w)) \nabla \hat{v}(S;w)(-1)\right)$$

$$= -\mathbb{E}\left((v_\pi(S) - \hat{v}(S;w)) \nabla \hat{v}(S;w)\right)$$

Value function methods: Optimization algorithms

El algoritmo de gradient descent en este caso es:

$$\boxed{w_{k+1} = w_k + \alpha_k \mathbb{E}\Big[(v_\pi(S) - \hat{v}(S; w_k))\nabla_w \hat{v}(S; w_k)\Big]}$$

- Esta expresión requiere calcular el valor esperado (también se requiere conocer la distribución de S).
- Podemos reemplazarlo utilizando un gradiente estocástico!
- Reemplazamos k por t para unificar la notación.

$$w_{t+1} = w_t + \alpha_t \Big(v_\pi(s_t) - \hat{v}(s_t; w_t)\Big)\nabla_w \hat{v}(s_t; w_t),$$

donde s_t es una muestra de $\mathcal{S}$ a tiempo t.

Value function methods: Optimization algorithms

$$w_{t+1} = w_t + \alpha_t \Big(v_\pi(s_t) - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)$$

Esta expresión todavía tiene un problema ya que depende de v_π

- **Monte Carlo**: Dado un episodio $(s_0, r_1, s_1, r_2, \dots)$, definimos g_t como el retorno (con descuento) a partir de s_t. Entonces g_t se puede usar como aproximación de $v_\pi(s_t)$:

$$w_{t+1} = w_t + \alpha_t \Big(g_t - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)$$

- **Temporal-difference**: En el espíritu de TD learning, podemos usar $r_{t+1} + \gamma \hat{v}(s_{t+1}; w_t)$ para aproximar $v_\pi(s_t)$:

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{v}(s_{t+1}; w_t) - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)$$

Value function methods: Optimization algorithms

Temporal-differences nos permite aproximar la **función de valor** para cada estado s por medio de $\hat{v}(s; w)$:

$$\boxed{w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{v}(s_{t+1}; w_t) - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)}$$

- En cada paso t requiere $\{(s_t, r_{t+1}, s_{t+1})\}$
- Solamente estima la función de valor para una política dada
- De alguna forma tratamos de resolver la ecuación de Bellman (Bellman expectation equation)

$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s]$$

- No estima el valor de las acciones ni calcula políticas óptimas

Value function methods: Neural network approximation example

Ver código
`Example_02_value_function_approx_with_nn.ipynb` Este código aproxima la función de valor v_π usando TD:

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{v}(s_{t+1}; w_t) - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)$$

Tareas:

- Revisar, ejecutar y comprender el código.
- En la sección 5, estudiar la influencia del número de neuronas.
- Comparar el *mean absolute error* (MAE) para distintas configuraciones.
- Comparar con el caso de la sección 8: Extension: one-hot input (tabular special case)

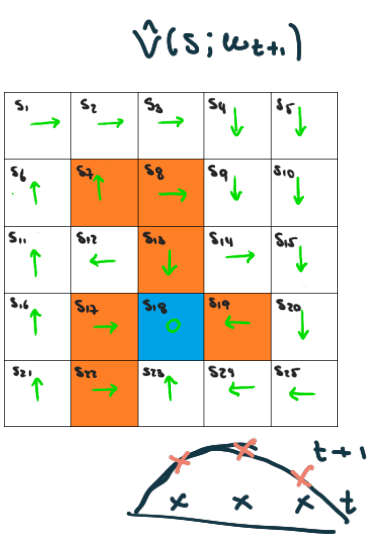
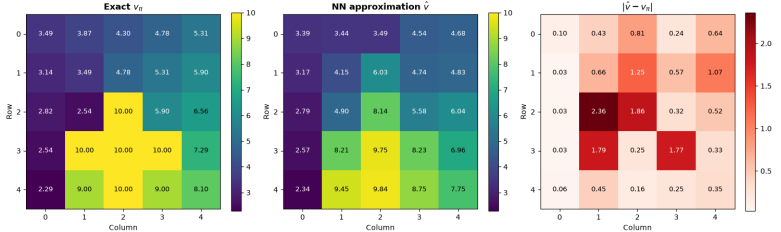
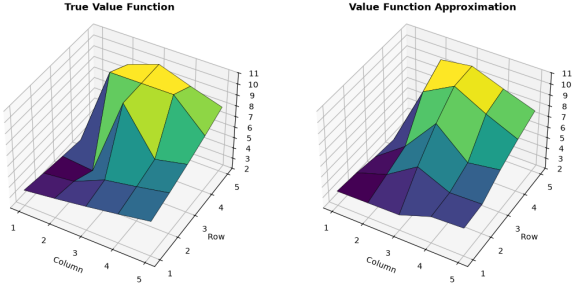


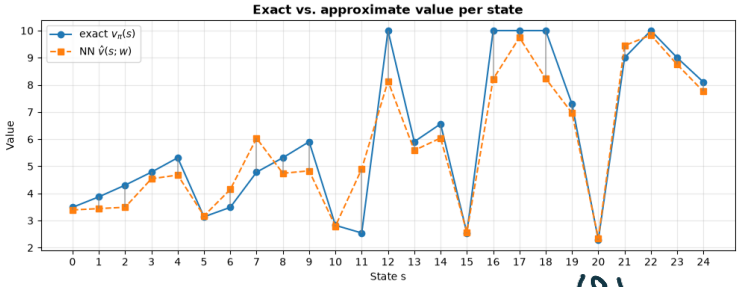
Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.

Value function methods: Neural network approximation example





Value function methods: Neural network approximation example



$\phi(S) = (f, c)$

$S_1 \rightarrow (0,0)$

$S_2 \rightarrow (0,1)$

$\hat{v}(S; w)$

$S_i = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$

$2 \times 10 + 10 \times 1 + 1$

$10 \times 10 + 10 + 1$

121

$$\boxed{w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{v}(s_{t+1}; w_t) - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)}$$

Cuando consideramos una aproximación lineal

$$\begin{aligned} \hat{v}(s; w) &= w_1 \phi_1(s) + w_2 \phi_2(s) + \cdots + w_d \phi_d(s) \\ &= w^\top \phi(s) = \phi(s)^\top w \end{aligned}$$

tenemos que:

$$\boxed{w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \phi(s_{t+1})^\top w_t - \phi(s_t)^\top w_t \Big) \phi(s_t)}$$ TD-linear

Obs: El caso lineal es más simple de estudiar y se conocen bien sus propiedades!

See Section 8.2, Zhao and Sections 9.4-9.5, Sutton & Barto.

Conexión con el caso tabular:

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{v}(s_{t+1}; w_t) - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)$$

$\phi(s) = e_s = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \in \mathbb{R}^n$

$\hat{v}(s; w) = e_s^\top w =: w(s)$

$\underbrace{[0 \dots 1 \dots 0]}_{e_{s_i}^\top} \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} = w_i = w(s_i)$

$w_{t+1} = w_t + \alpha_t (r_{t+1} + \gamma e_{s_{t+1}}^\top w_t - e_{s_t}^\top w_t) e_{s_t}$

Conexión con el caso tabular:

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{v}(s_{t+1}; w_t) - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)$$

$$w_{t+1} = w_t + \alpha_t \left(r_{t+1} + \gamma\, e_{s_{t+1}}^\top w_t - e_{s_t}^\top w_t \right) e_{s_t} \quad / \, e_{s_t}^\top$$

$$e_{s_t}^\top w_{t+1} = e_{s_t}^\top w_t + \alpha_t \left(r_{t+1} + \gamma\, e_{s_{t+1}}^\top w_t - e_{s_t}^\top w_t \right) e_{s_t}^\top \cdot e_{s_t}$$

$$w_{t+1}(s_t) = w_t(s_t) + \alpha_t \left(r_{t+1} + \gamma\, w_t(s_{t+1}) - w_t(s_t) \right) 1$$

$$\boxed{w_{t+1}(s_t) = w_t(s_t) + \alpha_t \left(r_{t+1} + \gamma\, w_t(s_{t+1}) - w_t(s_t) \right)}$$

Value function methods: Semi-gradient methods

$$J(w) = \frac{1}{2}\mathbb{E}\left[\left(v_\pi(s) - \hat{v}(s;w)\right)^2\right]$$

$$\downarrow$$

$$w_{t+1} = w_t + \alpha_t\Big(v_\pi(s_t) - \hat{v}(s_t;w_t)\Big)\nabla_w \hat{v}(s_t;w_t)$$

$$\downarrow$$

$$w_{t+1} = w_t + \alpha_t\Big(r_{t+1} + \gamma\hat{v}(s_{t+1};w_t) - \hat{v}(s_t;w_t)\Big)\nabla_w \hat{v}(s_t;w_t)$$

↳ viene de un funcional distinto

"semi-gradient methods"

Value function methods: Objective functions

Nota sobre las funciones objetivos

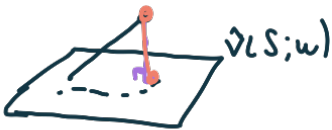
- **True value error**:

$$J_{\mathrm{E}}(w) = \mathbb{E}\Big[(v_\pi(S) - \hat{v}(S;w))^2\Big] = \|v_\pi - \hat{v}(w)\|_D^2,$$

donde $\|x\|_D^2 = x^\top D x$ y $D = \begin{bmatrix} \ddots & & \\ & d_\pi(s) & \\ & & \ddots \end{bmatrix}$.

- **Bellman error**:

$$J_{\mathrm{BE}}(w) = \|\hat{v}(w) - \underbrace{(r + \gamma P_\pi \hat{v}(w))}_{T_\pi(\hat{v}(w))}\|_D^2$$



- **Projected Bellman error**:

$$J_{\mathrm{PBE}}(w) = \|\hat{v}(w) - M T_\pi(\hat{v}(w))\|_D^2$$

Value function methods: Sarsa

- Hasta ahora hemos estimado la función de valor $v_\pi(s)$ mediante una función paramétrica $v(s, w)$.
- Para calcular políticas óptimas, debemos estimar el valor de las acciones en cada estado $q_\pi(s, a)$.
- Queremos entonces, $q_\pi(s, a) \approx \hat{q}(s, a; w)$
- Tenemos dos algoritmos de TD learning que podemos utilizar!

Value function methods: Sarsa

SARSA nos permite estimar el **valor de la acción** por medio del algoritmo de TD:

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}; w_t) - \hat{q}(s_t, a_t; w_t) \Big) \nabla_w \hat{q}(s_t, a_t; w_t)$$

donde $t = 0, 1, 2, \ldots$

- En cada paso t requiere $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}$
- Calcula el valor de la acción para una política dada aproximando la solución de la siguiente ecuación de Bellman:

$$q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

- Junto a un paso de mejora de política **puede calcular políticas óptimas**

TD methods: Learning action values (SARSA)

Algorithm 8.2: Sarsa with function approximation

Initialization: Initial parameter w_0. Initial policy π_0. $\alpha_t = \alpha > 0$ for all t. $\epsilon \in (0, 1)$.
Goal: Learn an optimal policy that can lead the agent to the target state from an initial state s_0.
For each episode, do
Generate a_0 at s_0 following $\pi_0(s_0)$
If s_t ($t = 0, 1, 2, \dots$) is not the target state, do
Collect the experience sample $(r_{t+1}, s_{t+1}, a_{t+1})$ given (s_t, a_t): generate r_{t+1}, s_{t+1} by interacting with the environment; generate a_{t+1} following $\pi_t(s_{t+1})$.
Update q-value:

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t)$$

Update policy:

$$\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|}(|\mathcal{A}(s_t)| - 1) \text{ if } a = \arg\max_{a \in \mathcal{A}(s_t)} \hat{q}(s_t, a, w_{t+1})$$

$$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}(s_t)|} \text{ otherwise}$$

$s_t \leftarrow s_{t+1}$, $a_t \leftarrow a_{t+1}$

Page 180, Zhao, Mathematical foundations of reinforcement learning

Value function methods: Q-learning

Q-learning nos permite estimar la **acciones óptimas** por medio del algoritmo de TD:

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \max_{s' \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a'; w_t) - \hat{q}(s_t, a_t; w_t) \Big) \nabla_w \hat{q}(s_t, a_t; w_t)$$

donde $t = 0, 1, 2, \ldots$

- En cada paso t requiere $\{(s_t, a_t, r_{t+1}, s_{t+1})\}$
- Calcula el valor de acciones óptimas aproximando la solución de la siguiente ecuación optimal de Bellman:

$$q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q_\pi(S_{t+1}, a') | S_t = s, A_t = a]$$

- No necesita de un paso de mejora de política para calcular políticas óptimas

Deep Q-learning

En `https://github.com/ibrevis/RL_unisabana_2026_2`,

- Ir a `/Lesson10/`
- Abrir `01_CartPole_intro.ipynb` para una introducción guiada al entorno
- Ir a `Example_03_deep_SARSA_CartPole.ipynb` para entrenar la política.
- Comprueba tues resultados con `run_deep_sarsa_cartpole_policy.py`.
- Estudiar qué sucede si modificamos el "semi-gradient"

